Assignment 2: Colab Fine-Tuning Notebook

This notebook is self-contained. Upload only this file to Google Colab, switch the runtime to `GPU`, and run the cells in order.

It will:

- create the custom e-commerce support dataset
- fine-tune `TinyLlama/TinyLlama-1.1B-Chat-v1.0` with LoRA
- evaluate the base model and fine-tuned model
- save artifacts to Google Drive if enabled

1. Mount Drive And Create Workspace

Set `USE_DRIVE = True` if you want the dataset and model artifacts to persist in Drive.

```
from pathlib import Path
import os

USE_DRIVE = True

if USE_DRIVE:
    from google.colab import drive
    drive.mount('/content/drive')
    PROJECT_DIR = Path('/content/drive/MyDrive/assignment_2_ecommer
else:
    PROJECT_DIR = Path('/content/assignment_2_ecommerce_finetuning'

for subdir in ['scripts', 'data/ecommerce_support', 'artifacts']:
    (PROJECT_DIR / subdir).mkdir(parents=True, exist_ok=True)

os.chdir(PROJECT_DIR)
print('Working directory:', PROJECT_DIR)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remo
Working directory: /content/drive/MyDrive/assignment_2_ecommerce_fir
```

2. Install Dependencies

```
!pip -q install torch transformers datasets accelerate peft sentenc
```

3. Generate The Custom Dataset

This creates a custom synthetic dataset with `720` examples and train/validation/test splits.

```
from __future__ import annotations

import csv
import json
import random
import re
from collections import Counter, defaultdict
from pathlib import Path

SEED = 42
STORE_NAME = 'ShopSphere'
EXAMPLES_PER_CATEGORY = 40
TRAIN_PER_CATEGORY = 32
VAL_PER_CATEGORY = 4
TEST_PER_CATEGORY = 4
OUTPUT_DIR = PROJECT_DIR / 'data' / 'ecommerce_support'

INTRO_PHRASES = ['Hi,', 'Hello,', 'Hey support,', 'Good morning,',
ENDINGS = ['Please help.', 'What should I do?', 'Please advise.', '
CUSTOMER_TYPES = ['I am a first-time customer.', 'I order from your
PRODUCTS = ['wireless earbuds', 'gaming mouse', 'Bluetooth speaker'
SIZES = ['XS', 'S', 'M', 'L', 'XL', 'UK 7', 'UK 8', 'UK 9', '128GB'
COLORS = ['black', 'blue', 'green', 'pink', 'white', 'gray', 'navy'
PAYMENT_METHODS = ['UPI', 'credit card', 'debit card', 'net banking
COUPONS = ['SAVE10', 'WELCOME15', 'FESTIVE20', 'APPONLY5', 'STYLE25
ISSUES = ['the app keeps showing an error', 'the tracking page has
TIME_PHRASES = ['yesterday', 'two days ago', 'three days ago', 'las
DELIVERY_WINDOWS = ['the promised delivery date was yesterday', 'it
RETURN_REASONS = ['I changed my mind', 'the product does not match
LOGIN_ISSUES = ['I am not receiving the OTP', 'the password reset l

CATEGORY_SPECS = [
    {
```

```
            'name': 'order_tracking',
            'prompts': [
                'I placed an order for a {product} and I want to track
                'Can you tell me the current status of order {order_id}
                'I ordered a {product} {time_phrase} and I want to know
                'I need a tracking update for order {order_id} because
            ],
            'responses': [
                "Please open My Orders and use the tracking link to see
                'You can track the shipment from the order details page
                'The fastest way to check the package status is from th
            ]
        },
        {
            'name': 'delayed_delivery',
            'prompts': [
                'My order for a {product} is delayed and {delivery_wind
                'Order {order_id} has not arrived yet even though {deli
                'I was expecting my {product} by now, but the parcel st
                'My shipment looks stuck in transit and {delivery_windo
            ],
            'responses': [
                'I am sorry your order is delayed. Please check the lat
                'Please review the courier tracking page from My Orders
                'A short delay can happen in transit, but the order sho
            ]
        },
        {
            'name': 'cancel_before_shipment',
            'prompts': [
                'I want to cancel my order for the {product} before it
                'Can I cancel order {order_id} right now? It still look
                'I placed a {product} order by mistake and I need to ca
                'My order is still in processing status. How do I cance
            ],
            'responses': [
                'If the order status is still Confirmed or Processing,
                'You can cancel the order yourself from the order detai
                'Cancellation is available only before shipment. Please
            ]
        },
        {
            'name': 'cancel_after_shipment',
            'prompts': [
                'My order for the {product} has already shipped, but I
                'Can you cancel order {order_id} even though the tracki
                'I no longer want the {product}, but it is already out
                'The order is already in transit and I need to cancel i
```

```
            ],
            'responses': [
                'Once an order has been shipped, it cannot be canceled
                'Shipped orders cannot be canceled anymore. If you do n
                'Cancellation is not available after shipment. Please r
            ]
        },
        {
            'name': 'return_request',
            'prompts': [
                'I received my {product}, but {return_reason} and I wan
                'How can I return order {order_id}? The item was delive
                'I want to place a return request for a {product} that
                'Please tell me the return process for my {product}.'
            ],
            'responses': [
                'You can request a return from My Orders within 7 days
                'Open the delivered order in My Orders and select Retur
                'Please raise the return request from your order detail
            ]
        },
        {
            'name': 'refund_status',
            'prompts': [
                'I returned order {order_id} and I want to know when my
                'My return pickup is complete, but I still do not see t
                'I am checking the refund status for a returned item. T
                'The return was approved a few days ago. How long does
            ],
            'responses': [
                'Refunds are processed after the returned item passes i
                'Once the return is verified, the refund is started aut
                'Please allow the return quality check to finish first.
            ]
        },
        {
            'name': 'exchange_request',
            'prompts': [
                'I ordered a {product} in size {size}, but I need size
                'I want to exchange my {color} {product} for {color_2}.
                'My {product} was delivered, but I need a different siz
                'Can you help me exchange order {order_id} for another
            ],
            'responses': [
                'You can request an exchange from My Orders within 7 da
                'Please open the delivered order and check whether Exch
                'Exchange requests are allowed within 7 days of deliver
            ]
```

```
        },
        {
            'name': 'wrong_item_received',
            'prompts': [
                'I ordered a {product}, but I received a {product_2} in
                'Order {order_id} contains the wrong item. What should
                'The package delivered today has a different product th
                'I received the wrong variant in my order and I need th
            ],
            'responses': [
                'I am sorry you received the wrong item. Please report
                'Please raise a Wrong Item issue within 48 hours of del
                'If the delivered item does not match the order, report
            ]
        },
        {
            'name': 'damaged_item',
            'prompts': [
                'My {product} arrived damaged today.',
                'Order {order_id} was delivered, but the item is broken
                'I opened my package and the {product} is damaged. How
                'The product I received has visible damage and I need a
            ],
            'responses': [
                'I am sorry the item arrived damaged. Please report the
                'Please open the order details page and submit a Damage
                'Damage issues must be reported within 48 hours of deli
            ]
        },
        {
            'name': 'missing_item',
            'prompts': [
                'My package was delivered, but the {product} is missing
                'Order {order_id} arrived with one item missing.',
                'I received the parcel today, but one product is not in
                'The box looked incomplete and my item is missing. What
            ],
            'responses': [
                'Please report the missing item within 48 hours of deli
                'If an item is missing from a delivered order, raise th
                'A missing item complaint should be submitted within 48
            ]
        },
        {
            'name': 'payment_failed',
            'prompts': [
                'My payment failed while using {payment_method}, and th
                'I was trying to place an order, but the checkout faile
```

```
                'The app showed a payment failure message when I tried
                'I cannot complete my purchase because the payment page
            ],
            'responses': [
                'Please try placing the order again after checking your
                'A failed payment usually means the order was not place
                'If the payment failed and no amount was deducted, you
            ]
        },
        {
            'name': 'payment_deducted_no_order',
            'prompts': [
                'My payment was deducted through {payment_method}, but
                'I paid for checkout, but the app crashed and I cannot
                'Money has been debited from my account, but I never go
                'The payment went through, but the order page is empty.
            ],
            'responses': [
                'If the payment was deducted but the order was not crea
                'Please wait for 3 to 5 business days because most fail
                'A deduction without order confirmation is normally ref
            ]
        },
        {
            'name': 'address_change',
            'prompts': [
                'I entered the wrong shipping address for order {order_
                'I want to update my delivery address before the order
                'The pin code on my order is wrong and I need to correc
                'Can I edit the address for my current order? It still
            ],
            'responses': [
                'You can change the delivery address only before the or
                'Address updates are allowed only until shipment. Open
                'Please try updating the address from My Orders while t
            ]
        },
        {
            'name': 'coupon_not_working',
            'prompts': [
                'The coupon code {coupon} is not working on my cart.',
                'I am trying to use {coupon}, but checkout says the cou
                'My discount code is failing even though I typed it cor
                'Why is the coupon not applying to order {order_id}?'
            ],
            'responses': [
                'Please check whether the coupon is still valid, meets
                'Coupon errors usually happen when the code is expired,
```

```
                'Please verify the coupon expiry, minimum spend, and pr
            ]
        },
        {
            'name': 'warranty_claim',
            'prompts': [
                'My {product} has stopped working and I want to claim w
                'How do I start a warranty claim for order {order_id}?'
                'The product was delivered earlier, but now it has deve
                'I need support for a warranty issue with my {product}.
            ],
            'responses': [
                'Please contact support with your order ID, a short des
                'To begin a warranty claim, share the order ID, product
                'Please keep the invoice and order ID ready and contact
            ]
        },
        {
            'name': 'out_of_stock_query',
            'prompts': [
                'The {product} I want to buy is out of stock. When will
                'Can you tell me if the {product} will be restocked soo
                'I am waiting for a {product}, but it still shows out o
                'Do you reserve stock for customers, or should I wait f
            ],
            'responses': [
                'Restock timing depends on the seller and current inven
                'Out-of-stock items are restocked based on seller inven
                'We cannot confirm a restock date in advance because av
            ]
        },
        {
            'name': 'account_login_issue',
            'prompts': [
                'I cannot log in to my account because {login_issue}.',
                'I am locked out of my shopping account and need help g
                'The app is not letting me sign in even though I am usi
                'How do I recover access to my account if login is fail
            ],
            'responses': [
                'Please use Forgot Password or the OTP login option fir
                'Start with password reset or OTP login using your regi
                'Please confirm that you are using the correct register
            ]
        },
        {
            'name': 'invoice_request',
            'prompts': [
```

```
                'I need the invoice for my {product} order.',
                'How can I download the bill for order {order_id}?',
                'I need a GST invoice or order invoice for reimbursemen
                'The item was delivered, but I cannot find the invoice
            ],
            'responses': [
                'You can download the invoice from My Orders once the i
                'Please check the order details page in My Orders for t
                'Invoices are usually available from the order details
            ]
        }
    ]

    def clean_text(text: str) -> str:
        text = re.sub(r'\s+', ' ', text).strip()
        text = text.replace(' ,', ',').replace(' .', '.').replace(' ?',
        return text

    def order_id(rng: random.Random) -> str:
        return f'SS{rng.randint(100000, 999999)}'

    def transaction_id(rng: random.Random) -> str:
        return f'TXN{rng.randint(10000000, 99999999)}'

    def prompt_with_style(rng: random.Random, body: str) -> str:
        parts = [rng.choice(INTRO_PHRASES), body, rng.choice(CUSTOMER_T
        return clean_text(' '.join(part for part in parts if part))

    def build_vars(rng: random.Random) -> dict:
        return {
            'product': rng.choice(PRODUCTS),
            'product_2': rng.choice(PRODUCTS),
            'order_id': order_id(rng),
            'transaction_id': transaction_id(rng),
            'time_phrase': rng.choice(TIME_PHRASES),
            'issue': rng.choice(ISSUES),
            'delivery_window': rng.choice(DELIVERY_WINDOWS),
            'return_reason': rng.choice(RETURN_REASONS),
            'payment_method': rng.choice(PAYMENT_METHODS),
            'size': rng.choice(SIZES),
            'size_2': rng.choice(SIZES),
            'color': rng.choice(COLORS),
            'color_2': rng.choice(COLORS),
            'coupon': rng.choice(COUPONS),
            'login_issue': rng.choice(LOGIN_ISSUES),
        }

    rng = random.Random(SEED)
```

```
    examples = []

    for spec in CATEGORY_SPECS:
        category_rows = []
        seen_prompts = set()
        while len(category_rows) < EXAMPLES_PER_CATEGORY:
            values = build_vars(rng)
            prompt_body = rng.choice(spec['prompts']).format(**values)
            prompt = prompt_with_style(rng, prompt_body)
            if prompt.lower() in seen_prompts:
                continue
            seen_prompts.add(prompt.lower())
            category_rows.append({
                'category': spec['name'],
                'prompt': prompt,
                'response': rng.choice(spec['responses']),
            })
        examples.extend(category_rows)

    grouped = defaultdict(list)
    for row in examples:
        grouped[row['category']].append(row)

    splits = {'train': [], 'validation': [], 'test': []}
    example_id = 1
    for category in sorted(grouped):
        rows = grouped[category][:]
        rng.shuffle(rows)
        split_map = {
            'train': rows[:TRAIN_PER_CATEGORY],
            'validation': rows[TRAIN_PER_CATEGORY:TRAIN_PER_CATEGORY +
            'test': rows[TRAIN_PER_CATEGORY + VAL_PER_CATEGORY:],
        }
        for split_name, split_rows in split_map.items():
            for row in split_rows:
                splits[split_name].append({
                    'id': f'ecs_{example_id:04d}',
                    'dataset_name': 'ecommerce_customer_support_assista
                    'domain': 'e-commerce customer support',
                    'store_name': STORE_NAME,
                    'split': split_name,
                    **row,
                })
                example_id += 1

    splits['all'] = splits['train'] + splits['validation'] + splits['te

    def write_jsonl(path: Path, rows: list[dict]) -> None:
```

```
    with path.open('w', encoding='utf-8') as handle:
        for row in rows:
            handle.write(json.dumps(row, ensure_ascii=True) + '\n')

def write_csv(path: Path, rows: list[dict]) -> None:
    if not rows:
        return
    with path.open('w', encoding='utf-8', newline='') as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].key
        writer.writeheader()
        writer.writerows(rows)

for split_name in ['all', 'train', 'validation', 'test']:
    write_jsonl(OUTPUT_DIR / f'{split_name}.jsonl', splits[split_na
    write_csv(OUTPUT_DIR / f'{split_name}.csv', splits[split_name])

stats = {
    'dataset_name': 'ecommerce_customer_support_assistant',
    'store_name': STORE_NAME,
    'total_examples': len(splits['all']),
    'split_counts': {name: len(splits[name]) for name in ['train',
    'category_counts': dict(Counter(row['category'] for row in spli
}
(OUTPUT_DIR / 'stats.json').write_text(json.dumps(stats, indent=2),

print(json.dumps(stats, indent=2))
print('Dataset directory:', OUTPUT_DIR)
```

```
{
  "dataset_name": "ecommerce_customer_support_assistant",
  "store_name": "ShopSphere",
  "total_examples": 720,
  "split_counts": {
    "train": 576,
    "validation": 72,
    "test": 72
  },
  "category_counts": {
    "account_login_issue": 40,
    "address_change": 40,
    "cancel_after_shipment": 40,
    "cancel_before_shipment": 40,
    "coupon_not_working": 40,
    "damaged_item": 40,
    "delayed_delivery": 40,
    "exchange_request": 40,
    "invoice_request": 40,
    "missing_item": 40,
    "order_tracking": 40,
```

```
    "out_of_stock_query": 40,
    "payment_deducted_no_order": 40,
    "payment_failed": 40,
    "refund_status": 40,
    "return_request": 40,
    "warranty_claim": 40,
    "wrong_item_received": 40
  }
}
Dataset directory: /content/drive/MyDrive/assignment_2_ecommerce_fir
```

```
import pandas as pd

train_df = pd.read_json(PROJECT_DIR / 'data' / 'ecommerce_support'
display(train_df.head(5))
print('Train rows:', len(train_df))
print('Unique categories:', train_df['category'].nunique())
```

	id	dataset_name	domain	store_name	sp
0	ecs_0001	ecommerce_customer_support_assistant	e-commerce customer support	ShopSphere	t
			e-		

4. Fine-Tune With LoRA

This cell trains a small instruct model. If Colab memory is tight, reduce batch size or set `MAX_LENGTH = 384`.

```
from __future__ import annotations

import inspect
import json
import math
import random

import torch
from datasets import load_dataset
```

```
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer, DataCo

MODEL_NAME = 'TinyLlama/TinyLlama-1.1B-Chat-v1.0'
OUTPUT_DIR = PROJECT_DIR / 'artifacts' / 'tinyllama_ecommerce_lora'
TRAIN_FILE = PROJECT_DIR / 'data' / 'ecommerce_support' / 'train.jso
VALIDATION_FILE = PROJECT_DIR / 'data' / 'ecommerce_support' / 'vali
SYSTEM_PROMPT = 'You are a helpful e-commerce customer support assis
MAX_LENGTH = 384
SEED = 42

def set_seed(seed: int) -> None:
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def build_prompt(user_prompt: str, system_prompt: str) -> str:
    return f'<|system|>\n{system_prompt}\n<|user|>\n{user_prompt.str

def tokenize_example(example: dict, tokenizer, max_length: int, syst
    prompt_text = build_prompt(example['prompt'], system_prompt)
    response_text = example['response'].strip() + tokenizer.eos_toke
    prompt_tokens = tokenizer(prompt_text, add_special_tokens=False)
    response_tokens = tokenizer(response_text, add_special_tokens=Fa
    input_ids = prompt_tokens['input_ids'] + response_tokens['input_
    attention_mask = prompt_tokens['attention_mask'] + response_toke
    labels = [-100] * len(prompt_tokens['input_ids']) + response_tok
    return {
        'input_ids': input_ids[:max_length],
        'attention_mask': attention_mask[:max_length],
        'labels': labels[:max_length],
    }

def compute_metrics(eval_preds):
    logits, labels = eval_preds
    if isinstance(logits, tuple):
        logits = logits[0]
    predictions = torch.tensor(logits).argmax(dim=-1)
    labels = torch.tensor(labels)
    shifted_predictions = predictions[:, :-1].contiguous()
    shifted_labels = labels[:, 1:].contiguous()
    valid_mask = shifted_labels != -100
    if valid_mask.sum().item() == 0:
        return {'token_accuracy': 0.0}
    correct = ((shifted_predictions == shifted_labels) & valid_mask)
    total = valid_mask.sum().item()
    return {'token_accuracy': correct / total}
```

```
set_seed(SEED)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, use_fast=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = 'right'

torch_dtype = torch.float16 if torch.cuda.is_available() else None
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME, torch_dtype
model.config.use_cache = False

peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias='none',
    task_type='CAUSAL_LM',
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate_pr
)
model = get_peft_model(model, peft_config)
model.print_trainable_parameters()

dataset = load_dataset('json', data_files={'train': str(TRAIN_FILE),
tokenized = dataset.map(
    lambda example: tokenize_example(example, tokenizer=tokenizer, m
    remove_columns=dataset['train'].column_names,
)

training_kwargs = {
    'output_dir': str(OUTPUT_DIR),
    'learning_rate': 2e-4,
    'num_train_epochs': 2,
    'per_device_train_batch_size': 2,
    'per_device_eval_batch_size': 2,
    'gradient_accumulation_steps': 8,
    'weight_decay': 0.01,
    'warmup_ratio': 0.1,
    'logging_steps': 10,
    'eval_steps': 25,
    'save_steps': 25,
    'save_total_limit': 2,
    'load_best_model_at_end': True,
    'metric_for_best_model': 'eval_loss',
    'greater_is_better': False,
    'report_to': 'none',
    'remove_unused_columns': False,
```

```
    'fp16': torch.cuda.is_available(),
    'seed': SEED,
}

training_args_signature = inspect.signature(TrainingArguments.__init
if 'evaluation_strategy' in training_args_signature.parameters:
    training_kwargs['evaluation_strategy'] = 'steps'
elif 'eval_strategy' in training_args_signature.parameters:
    training_kwargs['eval_strategy'] = 'steps'

training_args = TrainingArguments(**training_kwargs)
trainer_kwargs = {
    'model': model,
    'args': training_args,
    'train_dataset': tokenized['train'],
    'eval_dataset': tokenized['validation'],
    'data_collator': DataCollatorForSeq2Seq(tokenizer=tokenizer, pad
    'compute_metrics': compute_metrics,
}
trainer_signature = inspect.signature(Trainer.__init__)
if 'tokenizer' in trainer_signature.parameters:
    trainer_kwargs['tokenizer'] = tokenizer
elif 'processing_class' in trainer_signature.parameters:
    trainer_kwargs['processing_class'] = tokenizer

trainer = Trainer(**trainer_kwargs)
train_result = trainer.train()
metrics = trainer.evaluate()
metrics['train_loss'] = train_result.training_loss
if 'eval_loss' in metrics:
    metrics['eval_perplexity'] = math.exp(metrics['eval_loss'])

trainer.save_model()
tokenizer.save_pretrained(OUTPUT_DIR)
(OUTPUT_DIR / 'metrics.json').write_text(json.dumps(metrics, indent=
print(json.dumps(metrics, indent=2))
print('Saved adapter to:', OUTPUT_DIR)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
  warnings.warn(
```

```
config.json: 100%                                   608/608 [00:00<00:00, 46.0kB/s]

tokenizer_config.json:        1.29k/? [00:00<00:00, 101kB/s]

tokenizer.json:        1.84M/? [00:00<00:00, 97.9MB/s]
```

```
tokenizer.json:        1.84M/? [00:00<00:00, 27.9MB/s]

tokenizer.model: 100%                                   500k/500k [00:00<00:00, 2.50MB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Ple
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthe

special_tokens_map.json: 100%                           551/551 [00:00<00:00, 47.0kB/s]

model.safetensors: 100%                                 2.20G/2.20G [00:33<00:00, 57.4MB/s]

Loading weights: 100%   201/201 [00:00<00:00, 380.65it/s, Materializing param=model.norm.weigh

generation_config.json: 100%                            124/124 [00:00<00:00, 4.58kB/s]

WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems
trainable params: 12,615,680 || all params: 1,112,664,064 || trainab

Generating train split:        576/0 [00:00<00:00, 3253.81 examples/s]

Generating validation split:        72/0 [00:00<00:00, 633.74 examples/s]

Map: 100%                                               576/576 [00:04<00:00, 149.14 examples/
                                                        s]

Map: 100%                                               72/72 [00:00<00:00, 837.43 examples/
                                                        s]

warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_
```

5. Compare Base Model And Fine-Tuned Model

```
import json
import re
from pathlib import Path

import pandas as pd
import torch
from peft import PeftModel
from tqdm import tqdm
from transformers import AutoModelForCausalLM, AutoTokenizer

TEST_FILE = PROJECT_DIR / 'data' / 'ecommerce_support' / 'test.json
BASE_OUTPUT = PROJECT_DIR / 'artifacts' / 'base_eval.json'
TUNED_OUTPUT = PROJECT_DIR / 'artifacts' / 'finetuned_eval.json'

def normalize_text(text: str) -> str:
    text = text.lower().strip()
    text = re.sub(r'[^a-z0-9\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
```

```
    return text

def exact_match(prediction: str, reference: str) -> float:
    return float(normalize_text(prediction) == normalize_text(refer

def token_f1(prediction: str, reference: str) -> float:
    pred_tokens = normalize_text(prediction).split()
    ref_tokens = normalize_text(reference).split()
    if not pred_tokens or not ref_tokens:
        return 0.0
    pred_counts = {}
    ref_counts = {}
    for token in pred_tokens:
        pred_counts[token] = pred_counts.get(token, 0) + 1
    for token in ref_tokens:
        ref_counts[token] = ref_counts.get(token, 0) + 1
    overlap = 0
    for token, count in pred_counts.items():
        overlap += min(count, ref_counts.get(token, 0))
    if overlap == 0:
        return 0.0
    precision = overlap / len(pred_tokens)
    recall = overlap / len(ref_tokens)
    return 2 * precision * recall / (precision + recall)

def build_prompt(user_prompt: str, system_prompt: str) -> str:
    return f'<|system|>\n{system_prompt}\n<|user|>\n{user_prompt.st

def load_rows(path: Path) -> list[dict]:
    rows = []
    with path.open('r', encoding='utf-8') as handle:
        for line in handle:
            if line.strip():
                rows.append(json.loads(line))
    return rows

def load_model(model_name: str, adapter_path: Path | None = None):
    tokenizer = AutoTokenizer.from_pretrained(str(adapter_path) if
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    torch_dtype = torch.float16 if torch.cuda.is_available() else N
    model = AutoModelForCausalLM.from_pretrained(model_name, torch_
    if adapter_path:
        model = PeftModel.from_pretrained(model, str(adapter_path))
    model.eval()
    return tokenizer, model

def generate_answer(tokenizer, model, prompt: str, max_new_tokens:
```

```
    inputs = tokenizer(prompt, return_tensors='pt').to(model.device
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )
    generated_ids = outputs[0][inputs['input_ids'].shape[1]:]
    return tokenizer.decode(generated_ids, skip_special_tokens=True

def evaluate_model(model_name: str, adapter_path: Path | None, outp
    rows = load_rows(TEST_FILE)
    tokenizer, model = load_model(model_name, adapter_path)
    exact_scores = []
    f1_scores = []
    predictions = []
    for row in tqdm(rows, desc=f'Evaluating {adapter_path.name if a
        prompt = build_prompt(row['prompt'], SYSTEM_PROMPT)
        prediction = generate_answer(tokenizer, model, prompt)
        em = exact_match(prediction, row['response'])
        f1 = token_f1(prediction, row['response'])
        exact_scores.append(em)
        f1_scores.append(f1)
        predictions.append({
            'id': row['id'],
            'category': row['category'],
            'prompt': row['prompt'],
            'reference': row['response'],
            'prediction': prediction,
            'exact_match': em,
            'token_f1': f1,
        })
    summary = {
        'model_name': model_name,
        'adapter_path': str(adapter_path) if adapter_path else None
        'examples_evaluated': len(predictions),
        'exact_match': sum(exact_scores) / len(exact_scores),
        'token_f1': sum(f1_scores) / len(f1_scores),
        'predictions': predictions,
    }
    output_path.write_text(json.dumps(summary, indent=2), encoding=
    return summary

base_summary = evaluate_model(MODEL_NAME, adapter_path=None, output
tuned_summary = evaluate_model(MODEL_NAME, adapter_path=OUTPUT_DIR,
```

```
print('Base metrics:', {k: v for k, v in base_summary.items() if k
print('Fine-tuned metrics:', {k: v for k, v in tuned_summary.items(

comparison_rows = []
for base_row, tuned_row in zip(base_summary['predictions'][:10], tu
    comparison_rows.append({
        'category': tuned_row['category'],
        'prompt': tuned_row['prompt'],
        'reference': tuned_row['reference'],
        'base_prediction': base_row['prediction'],
        'fine_tuned_prediction': tuned_row['prediction'],
    })

comparison_df = pd.DataFrame(comparison_rows)
display(comparison_df)
```

```
Loading weights: 100%   201/201 [00:09<00:00, 20.28it/s, Materializing param=model.norm.weight]

Evaluating base: 100%|██████████| 72/72 [1:12:18<00:00, 60.25s/it]
Loading weights: 100%   201/201 [00:07<00:00, 22.68it/s, Materializing param=model.norm.weight]

Evaluating tinyllama_ecommerce_lora: 100%|██████████| 72/72 [45:41<0
Fine-tuned metrics: {'model_name': 'TinyLlama/TinyLlama-1.1B-Chat-v1
```

	category	prompt	reference	base_prediction	fine_tuned
0	account_login_issue	Good evening, I am locked out of my shopping a...	Please use Forgot Password or the OTP login op...	As a first-time customer, you should follow th...	Plea Password or
1	account_login_issue	Good evening, I cannot log in to my account be...	Start with password reset or OTP login using y...	I am unable to access your account or provide ...	Plea Password or

Hi, How

```
import torch
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
ADAPTER_PATH = OUTPUT_DIR   # path where LoRA adapter is saved

SYSTEM_PROMPT = """You are a helpful e-commerce customer support as
Answer clearly and politely according to the store policy shown in
Give short practical next steps and do not invent unsupported polic

def build_prompt(user_prompt: str, system_prompt: str) -> str:
    return f"<|system|>\n{system_prompt}\n<|user|>\n{user_prompt.st

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(str(ADAPTER_PATH), use_fa

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Load base model
base_model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16 if torch.cuda.is_available() else Non
)

# Load LoRA adapter
model = PeftModel.from_pretrained(base_model, str(ADAPTER_PATH))
model.eval()

def ask_model(user_prompt: str, max_new_tokens: int = 120):
    prompt = build_prompt(user_prompt, SYSTEM_PROMPT)
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )

    generated_ids = outputs[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(generated_ids, skip_special_tokens=True

# Test prompt
test_prompt = "My payment was deducted but no order was created. Wh
print(ask_model(test_prompt))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.
The secret `HF_TOKEN` does not exist in your Colab secrets.
```

```
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Ple
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthe
```

Loading weights: 100% 201/201 [00:00<00:00, 409.33it/s, Materializing param=model.norm.weigh

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems
```

```
from pathlib import Path

PROJECT_DIR = Path("/content/drive/MyDrive/assignment_2_ecommerce_f
OUTPUT_DIR = PROJECT_DIR / "artifacts" / "tinyllama_ecommerce_lora"

MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
ADAPTER_PATH = OUTPUT_DIR
```

```
import torch
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
ADAPTER_PATH = OUTPUT_DIR   # path where LoRA adapter is saved

SYSTEM_PROMPT = """You are a helpful e-commerce customer support as
Answer clearly and politely according to the store policy shown in
Give short practical next steps and do not invent unsupported polic

def build_prompt(user_prompt: str, system_prompt: str) -> str:
    return f"<|system|>\n{system_prompt}\n<|user|>\n{user_prompt.st

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(str(ADAPTER_PATH), use_fa

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Load base model
base_model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16 if torch.cuda.is_available() else Non
)

# Load LoRA adapter
model = PeftModel.from_pretrained(base_model, str(ADAPTER_PATH))
```

```
model.eval()

def ask_model(user_prompt: str, max_new_tokens: int = 120):
    prompt = build_prompt(user_prompt, SYSTEM_PROMPT)
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )

    generated_ids = outputs[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(generated_ids, skip_special_tokens=True

# Test prompt
test_prompt = "I received the package today, but the box looks incc
print(ask_model(test_prompt))
```

```
Loading weights: 100%   201/201 [00:00<00:00, 264.80it/s, Materializing param=model.norm.weigh
```

```
import torch
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
ADAPTER_PATH = OUTPUT_DIR   # path where LoRA adapter is saved

SYSTEM_PROMPT = """You are a helpful e-commerce customer support ass
Answer clearly and politely according to the store policy shown in t
Give short practical next steps and do not invent unsupported polici

def build_prompt(user_prompt: str, system_prompt: str) -> str:
    return f"<|system|>\n{system_prompt}\n<|user|>\n{user_prompt.str

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(str(ADAPTER_PATH), use_fas

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Load base model
base_model = AutoModelForCausalLM.from_pretrained(
```

```
    MODEL_NAME,
    torch_dtype=torch.float16 if torch.cuda.is_available() else None
)

# Load LoRA adapter
model = PeftModel.from_pretrained(base_model, str(ADAPTER_PATH))
model.eval()

def ask_model(user_prompt: str, max_new_tokens: int = 120):
    prompt = build_prompt(user_prompt, SYSTEM_PROMPT)
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )

    generated_ids = outputs[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(generated_ids, skip_special_tokens=True)

# Test prompt
test_prompt = "The coupon code SAVE10 is not applying to my cart. Wh
print(ask_model(test_prompt))
```

```
Loading weights: 100%   201/201 [00:00<00:00, 522.77it/s, Materializing param=model.norm.weigh
```